

CRISISREADY

Trening reakcji kryzysowych w Mixed Reality oparty na mikroprogresji

Sprawozdanie z projektu zaliczeniowego z Technologii Kognitywnych

Zespół:

Wiktoria Bartek

Jakub Cieniuch

Magdalena Grzesiak

Oliwia Frueauff

Adriana Jankowiak

09.06.2026

Spis treści

Spis treści	3
1. Streszczenie	5
2. Wprowadzenie: problem, kontekst i cel projektu	6
3. Uzasadnienie kognitywne i odniesienie do artykułu wzorcowego	7
4. Pytania badawcze, hipotezy i kryteria sukcesu	8
5. Koncepcja systemu i scenariusz użytkownika	9
6. Mikroprogresja i mapa kroków	10
7. Pętla poznawcza systemu: SENSE – INTERPRET – DECIDE – ACT – FEEDBACK..	12
8. UX/UI i projektowanie interakcji kryzysowej	13
9. Architektura techniczna	15
10. Wykorzystane technologie, narzędzia i zasoby	16
11. Funkcjonalność programu	17
12. Logowanie danych i przygotowanie walidacji	18
13. Proces wytwórczy, harmonogram i metodyka pracy	19
14. Podział obowiązków w zespole	21
15. Testowanie techniczne	22
16. Propozycja badania użytkowników i procedura walidacji	23
17. Metryki efektywności, UX i obciążenia poznawczego	24
18. Wyzwania etyczne, prywatność i bezpieczeństwo	25
19. Wykorzystanie AI.....	27
20. Screeny, diagramy i dokumentacja działania	28
21. Niepowodzenia, ograniczenia i wnioski z błędów	30
22. Upowszechnienie projektu	33
23. Dalszy rozwój systemu.....	34

24. Wnioski końcowe	36
Bibliografia.....	36

1. Streszczenie

CrisisReady jest prototypem aplikacji Mixed Reality/Virtual Reality przeznaczonej na gogle Meta Quest 3/3S. System realizuje trening reakcji w sytuacjach kryzysowych poprzez jedną ciągłą sesję fabularno-treningową, składającą się z trzech modułów:

- wyjścia z poślizgu samochodem (KIEROWNICA),
- komunikacji alarmowej z numerem 112 (MOWA) oraz
- zachowania ciszy podczas ukrycia przed napastnikiem (CISZA).

Projekt nie został zaprojektowany wyłącznie jako gra lub demonstracja VR, lecz jako technologia kognitywna – system zbiera dane o zachowaniu użytkownika, interpretuje je, podejmuje decyzje o przebiegu scenariusza, uruchamia konsekwencje działania i dostarcza feedback wspierający uczenie.

Główną ramą teoretyczną projektu jest **mikroprogresja**, rozumiana jako rozbicie złożonej umiejętności na krótkie, mierzalne i stopniowo integrowane kroki. Inspiracją do stworzenia projektu był artykuł *MR.Drum*, w którym zaproponowano wykorzystanie Mixed Reality do nauki złożonych koordynacyjnie umiejętności przez oddzielenie warstwy poznawczej, motorycznej i integracyjnej (Wang et al., 2025). W *CrisisReady* podobna logika została przeniesiona z nauki gry na perkusji na trening zachowania kryzysowego: użytkownik najpierw uczy się prostych reakcji, a następnie wykonuje je w zintegrowanym, stresującym scenariuszu.

Projekt osiągnął postać grywalnego prototypu działającego na fizycznym sprzęcie Meta Quest 3S. Zaimplementowano sceny MR/VR, menu, tryb treningowy, śledzenie dłoni, dialog głosowy, detekcję ciszy, detektory mikrokroków, system aktów, podstawowe logowanie metryk do pliku JSONL oraz strukturę kwestionariuszy badawczych.

Prototyp umożliwia przejście pełnej sekwencji: jazda - poślizg - wypadek - ukrycie - telefon 112 - cisza - wygrana lub przegrana.

Słowa kluczowe: Mixed Reality, Virtual Reality, Meta Quest 3/3S, mikroprogresja, obciążenie poznawcze, scaffolding, trening reakcji kryzysowych, technologia kognitywna, śledzenie dłoni, Wit.ai.

2. Wprowadzenie: problem, kontekst i cel projektu

Szkolenia dotyczące zachowania w sytuacjach kryzysowych są zwykle prowadzone w formie instruktaży, prezentacji lub ćwiczeń symulacyjnych. Takie formy mogą przekazywać wiedzę deklaratywną, lecz często nie uczą automatycznych odruchów potrzebnych w sytuacji presji czasu, wysokiego pobudzenia emocjonalnego i niepewności. Użytkownik może wiedzieć, co powinien zrobić, ale niekoniecznie potrafi wykonać to działanie w stresie, kiedy rośnie obciążenie poznawcze, a uwaga koncentruje się na najbardziej wyrazistych bodźcach.

Mixed Reality i Virtual Reality pozwalają częściowo rozwiązać ten problem, ponieważ łączą bezpieczeństwo treningu z możliwością wywołania poczucia obecności, presji i konieczności działania. W *CrisisReady* realne otoczenie użytkownika, widoczne przez passthrough, zostaje włączone do scenariusza, a następnie zastąpione pełną sceną VR. Użytkownik nie tylko czyta instrukcje, ale wykonuje działanie ciałem, głosem i kontrolą zachowania.

Celem projektu było zaprojektowanie i zaimplementowanie prototypu systemu MR/VR, który wspiera trening reakcji kryzysowych przez mikroprogresję. System miał uczyć krótkich reakcji, mierzyć ich wykonanie oraz dostarczać feedback w taki sposób, aby nie zwiększać nadmiernie frustracji i obciążenia poznawczego. Projekt miał także przygotować podstawę do dalszej walidacji naukowej: logowania danych, badań UX, pomiaru obciążenia poznawczego i analizy efektywności uczenia.

Problem projektowy	Odpowiedź systemu <i>CrisisReady</i>
Trudności w treningu reakcji pod presją	Immersyjny scenariusz kryzysowy z konsekwencjami błędów
Nadmierna złożoność pełnych procedur	Rozbicie na mikrokroki i osobny tryb treningowy
Brak obiektywnego pomiaru postępu	Logowanie czasu, błędów, prób i ukończenia etapów
Ryzyko przeciążenia poznawczego	Krótki, kontekstowy feedback i ograniczona liczba komunikatów
Trudność transferu wiedzy do działania	Uczenie przez ruch, głos i kontrolę zachowania w scenariuszu

3. Uzasadnienie kognitywne i odniesienie do artykułu wzorcowego

Projekt został oparty na założeniu, że technologia kognitywna nie jest jedynie narzędziem prezentowania treści, lecz systemem wpływającym na percepcję, uwagę, decyzje, działanie, emocje i poczucie sprawczości użytkownika. W tym sensie **CrisisReady** nie jest tylko aplikacją szkoleniową w VR. Jest systemem zamkniętej pętli, w którym dane sensoryczne i behawioralne użytkownika wpływają na przebieg doświadczenia.

Artykuł *MR.Drum: Designing Mixed Reality Interfaces to Support Structured Learning Micro-Progression in Drumming* stanowił punkt odniesienia dla sposobu myślenia o projekcie (Wang et al., 2025). Autorzy pokazali, że złożoną umiejętność można rozłożyć na mikroetapy i wspierać ją w Mixed Reality przez stopniowanie poziomu trudności, wizualny feedback i integrację wielu kanałów działania. W **CrisisReady** przeniesiono tę ideę na domenę zachowań kryzysowych. Zamiast rytmu i koordynacji kończyn system uczy sekwencji reakcji: kontroli kierownicy, jasnej komunikacji oraz samoregulacji w sytuacji zagrożenia.

Drugim ważnym pojęciem jest scaffolding, czyli tymczasowe wsparcie udzielane użytkownikowi w trakcie nabywania umiejętności (Wood et al., 1976). W prototypie scaffolding pojawia się w trybie TRENING, w prostych instrukcjach oraz w podziale zadania na rozpoznawalne mikrokroki. Docelowo wsparcie powinno być stopniowo ograniczane, aby sprawdzić, czy użytkownik potrafi wykonać zadanie samodzielnie.

Trzecia rama teoretyczna dotyczy obciążenia poznawczego. W sytuacji kryzysowej użytkownik musi jednocześnie przetwarzać bodźce, podejmować decyzje, kontrolować działanie i regulować emocje. Zgodnie z teorią obciążenia poznawczego nadmiar informacji może utrudniać uczenie i wykonanie zadania (Sweller, 1988). Dlatego interfejs **CrisisReady** został pomyślany jako kryzysowy, a nie dekoracyjny: ma ograniczać liczbę komunikatów, skracać instrukcje i dostarczać feedback dokładnie wtedy, gdy jest potrzebny.

4. Pytania badawcze, hipotezy i kryteria sukcesu

Projekt był przede wszystkim prototypem implementacyjnym, ale od początku zakładał możliwość walidacji empirycznej. Pytania badawcze zostały sformułowane tak, aby można było je sprawdzić w przyszłym badaniu z użytkownikami, wykorzystując logi systemowe, kwestionariusze oraz obserwację zachowania.

Numer	Pytanie badawcze	Proponowany wskaźnik
PB1	Czy mikroprogresja w MR/VR skraca czas reakcji w scenariuszach kryzysowych?	czas do pierwszej poprawnej reakcji, czas ukończenia mikrokroku
PB2	Czy liczba błędów krytycznych spada po treningu?	liczba błędnych reakcji, liczba restartów, liczba podpowiedzi
PB3	Które mikrokroki są najtrudniejsze dla użytkowników?	najczęściej powtarzane lub nieukończone mikrokroki
PB4	Czy użytkownik po treningu potrzebuje mniej podpowiedzi?	liczba komunikatów pomocniczych na sesje
PB5	Czy nabyta umiejętność przenosi się na zmieniony wariant scenariusza?	wynik w teście transferu
PB6	Czy system poprawia wykonanie bez nadmiernego wzrostu obciążenia poznawczego?	Wynik NASA-TLX, czas, błędy i subiektywna frustracja
PB7	Czy interfejs jest zrozumiały pod presją czasu?	SUS, UEQ, obserwacje i wywiad po sesji

Hipoteza główna: trening z wykorzystaniem *CrisisReady* poprawi skuteczność wykonania mikrokroków kryzysowych w porównaniu z wykonaniem przed treningiem, przy utrzymaniu akceptowalnego poziomu obciążenia poznawczego i dyskomfortu.

Hipotezy szczegółowe zakładają, że po treningu użytkownicy będą szybciej reagować, popełniać mniej błędów, rzadziej korzystać z podpowiedzi oraz lepiej oceniać zrozumiałość zadania.

Kryterium sukcesu	Minimalny oczekiwany rezultat
Działanie techniczne	Prototyp uruchamia się na Quest 3S i pozwala przejść pełną sesję
Mikroprogresja	Każdy moduł posiada rozpoznawalne mikrokroki i kryteria ich wykonania
Closed-loop	System reaguje na dane użytkownika, a nie tylko odtwarza scenę
Walidacja	System zapisuje dane umożliwiające analizę po sesji
UX	Użytkownik rozumie cel zadania i potrafi uruchomić scenariusz

5. Koncepcja systemu i scenariusz użytkownika

CrisisReady zostało zaprojektowane jako ciągła sekwencja zdarzeń, a nie trzy niepowiązane minigry. Użytkownik rozpoczyna w menu, wybiera tryb gry lub treningu, następnie wchodzi w scenariusz prowadzący od jazdy samochodem przez wypadek do ukrycia się i kontaktu z numerem alarmowym. Taka struktura wzmacnia poczucie ciągłości i pozwala integrować trzy typy umiejętności: motoryczną, komunikacyjną i samoregulacyjną.



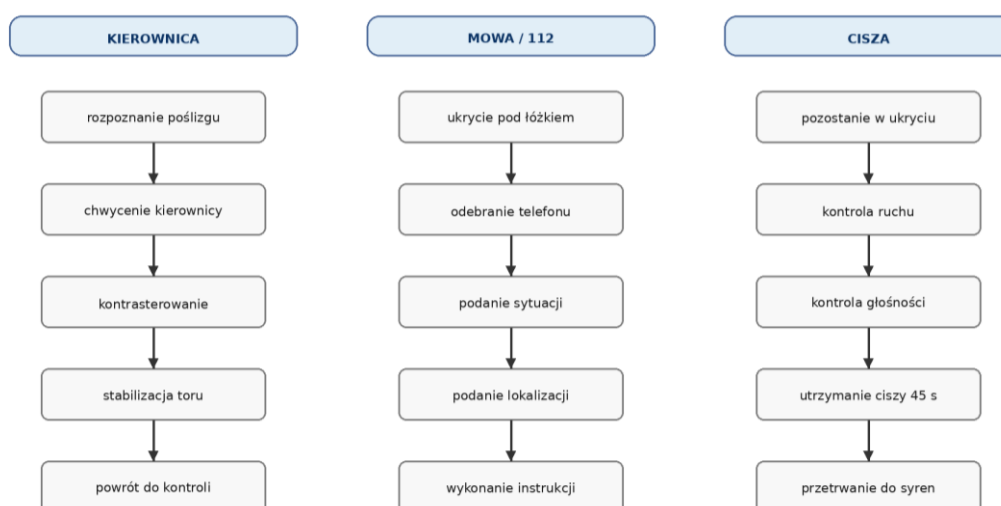
Scenariusz można streścić jako sekwencję: jazda samochodem - poślizg - wypadek - przebudzenie w sypialni - ukrycie pod łóżkiem - telefon 112 - zachowanie ciszy - przyjazd policji albo przegrana. W warstwie narracyjnej użytkownik przechodzi przez sytuację narastającego zagrożenia, natomiast w warstwie treningowej wykonuje trzy oddzielne zadania: kontrolę ruchu, komunikację i zahamowanie reakcji.

Faza scenariusza	Tryb środowiska	Zadanie użytkownika	Funkcja poznawcza
Menu i trening	MR/VR	Wybór GRAJ lub TRENING	Orientacja w interfejsie, rozumienie celu
Jazda	Passthrough + elementy VR	Chwyt kierownicy i reakcja na poślizg	Uwaga wzrokowa, koordynacja, decyzja motoryczna
Wypadek	VR / czarny ekran	Przejście narracyjne do kolejnego aktu	Przetwarzanie konsekwencji zdarzenia
Ukrycie	VR	Schylenie głowy i ukrycie się pod łóżkiem	Kontrola pozycji ciała, adaptacja do sytuacji
Telefon 112	VR + audio	Wypowiedzenie formułek alarmowych	Język, pamięć robocza, kontrola emocji
Cisza	VR + mikrofon	Utrzymanie ciszy przez 45 sekund	Samoregulacja, hamowanie reakcji, odporność na stres
Wynik	Ekran końcowy	Restart lub zakończenie	Feedback, uczenie się na błędzie

6. Mikroprogresja i mapa kroków

Mikroprogresja oznacza, że użytkownik nie jest od razu wrzucany w pełną, wieloetapową procedurę. Złożone działanie zostaje rozbite na mniejsze jednostki, które można osobno ćwiczyć, mierzyć i wzmacniać feedbackiem. W **CrisisReady** każdy z trzech modułów odpowiada innemu typowi umiejętności:

- KIEROWNICA trenuje reakcję motoryczną,
- MOWA trenuje uporządkowaną komunikację,
- CISZA trenuje samokontrolę.



Moduł	Mikrokrok	Dane wejściowe	Przykładowe zaliczenie kryterium
KIEROWNICA	Chwycenie kierownicy	Pozycja dłoni	Dłoń w obszarze kierownicy przez minimalny czas
KIEROWNICA	Kontrsterowanie	Obrót kierownicy/ gest	Skręt w odpowiednim kierunku w oknie czasowym
MOWA	Wypowiedź 112	Mikrofon	Wykrycie słów kluczowych lub formuлки
CISZA	Utrzymanie ciszy	Mikrofon/poziom hałasu	Brak przekroczenia progu głośności przez 45 sekund
CISZA	Kontrola bezruchu	Pozycja głowy/dłoni	Brak gwałtownych zmian w pozycji w czasie zagrożenia

Każdy mikrokrok może być oceniany w uproszczonej skali:

- 0 - brak wykonania lub działanie szkodliwe,
- 1 - wykonanie z pomocą albo po czasie,
- 2 - wykonanie samodzielne i poprawne.

Taka punktacja umożliwia budowę prostego indeksu mikroprogresji: suma punktów z mikrokroków pokazuje, czy użytkownik przechodzi od działania niepewnego do działania samodzielnego.

7. Pętla poznawcza systemu: SENSE – INTERPRET – DECIDE – ACT – FEEDBACK

Najważniejszym elementem uzasadnienia projektu jako technologii kognitywnej jest opis pętli poznawczej. *CrisisReady* nie działa jak statyczny film VR. System zbiera dane o użytkowniku, interpretuje je w kontekście scenariusza, decyduje, czy mikrokrok został wykonany poprawnie, uruchamia kolejne zdarzenie i przekazuje użytkownikowi informację zwrotną. Taka pętla pozwala traktować aplikację jako system wspierający decyzje i zachowania, a nie wyłącznie jako środowisko wizualne.

Etap pętli	Realizacja w <i>CrisisReady</i>	Znaczenie kognitywne
SENSE	hand tracking, mikrofon, pozycja głowy, passthrough/VR	system odbiera sygnały związane z percepcją, ruchem i zachowaniem
INTERPRET	detektory mikrokroków, Wit.ai, progowanie ciszy	system interpretuje zachowanie jako poprawne, częściowe lub błędne
DECIDE	ScenarioRunner, MicrostepProgressor, SessionFlowManager	system wybiera dalszy przebieg scenariusza na podstawie danych
ACT	zmiana aktu, dialog dyspozytora, jumpscare, ekran końcowy	system uruchamia konsekwencję działania użytkownika
FEEDBACK	komunikaty, dźwięki, konsekwencje, wynik	użytkownik otrzymuje informację wzmacniającą uczenie

Warto podkreślić, że feedback w *CrisisReady* nie jest długi ani nadmiernie opisowy. W sytuacji kryzysowej użytkownik nie ma zasobów poznawczych na czytanie rozbudowanych instrukcji. Feedback powinien być krótki, jednoznaczny i powiązany z aktualnym działaniem. Tę zasadę zastosowano w projekcie menu, treningu i komunikatów końcowych.

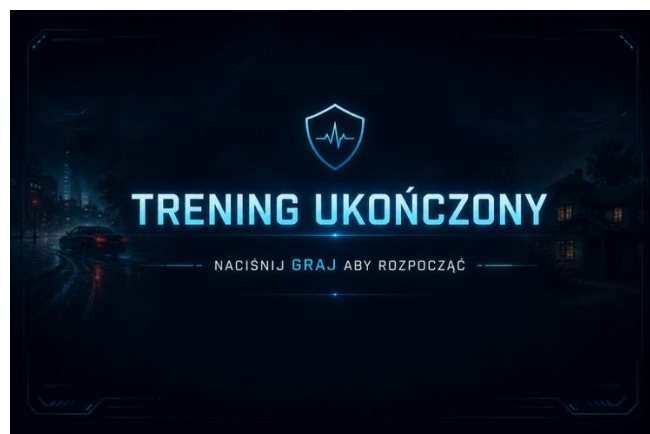
8. UX/UI i projektowanie interakcji kryzysowej

Interfejs **CrisisReady** został zaprojektowany zgodnie z zasadą: najpierw działanie, potem wyjaśnienie. Oznacza to, że w trakcie sytuacji kryzysowej system nie powinien zasypywać użytkownika instrukcjami, ikonami ani analizą wyników. Zadaniem UI jest ograniczenie obciążenia poznawczego i umożliwienie szybkiego rozpoznania, co należy zrobić, dlatego najważniejsze komunikaty są krótkie, a ćwiczenia nazwano prostymi hasłami: KIEROWNICA, MOWA i CISZA.

Projekt UI obejmował ekran startowy z nazwą **CrisisReady**, przyciski GRAJ i TRENING, ekran ukończenia treningu, komunikaty ćwiczeń oraz ekran przegranej z przyciskiem SPRÓBUJ PONOWNIE. W późniejszych iteracjach usunięto część zbędnych elementów graficznych i uproszczono napisy, aby interfejs był bardziej czytelny.

Zasada UX/UI	Zastosowanie w projekcie	Uzasadnienie
Minimalizm komunikatów	Krótkie nazwy modułów i interakcje	Redukcja obciążenia poznawczego
Jednoznaczność sygnałów	Każdy etap ma jasny cel	Mniejsze ryzyko pomylenia reakcji
Feedback kontekstowy	Reakcja systemu następuje po działaniu	Uczenie się przez konsekwencję
Dostępność interakcji	Sterowanie dłońmi, brak kontrolerów	Większa naturalność, ale także większe wyzwania techniczne
Tryb treningowy	Osobny tryb przed grą główną	Scaffolding i przygotowanie użytkownika





9. Architektura techniczna

Architektura systemu została zaprojektowana modułowo.

- Warstwa wejściowa odbiera dane z gogli, mikrofonu i systemu śledzenia dłoni.
- Warstwa interpretacyjna sprawdza, czy dane odpowiadają oczekiwanemu mikrokrokowi.
- Warstwa scenariusza sekwencjonuje akty i przejścia.
- Warstwa feedbacku uruchamia odpowiednie reakcje, a warstwa badawcza zapisuje logi do późniejszej analizy.



Kluczowe klasy i komponenty systemu obejmują:

- ScenarioConfig (ScriptableObject) – definiuje kroki i parametry scenariusza,
- ScenarioRunner – uruchamia kolejne mikrokroki,
- MicrostepProgressor – kontroluje postęp, zaliczenia i przejścia,
- SteeringRecoveryDetector – sprawdza reakcję w module poślizgu,
- UnderBedDetector – sprawdza, czy użytkownik znajduje się w pozycji ukrycia,
- PhoneAnswerDetector – wykrywa odebranie telefonu,
- WitDictationDetector – interpretuje wypowiedzi użytkownika,
- SilenceMicrostepDetector – kontroluje poziom hałasu w module CISZA,
- Act2DriveDirector – reżyseruje jazdę, poślizg i wypadek,
- Act3HidingDirector – reżyseruje pokój, telefon, dialog i ciszę,
- JSONLLogger – zapisuje metryki do analizy.

10. Wykorzystane technologie, narzędzia i zasoby

Projekt został zaimplementowany w Unity 6 LTS z wykorzystaniem C# i URP. Docelową platformą były gogle Meta Quest 3 / 3S, działające jako samodzielne urządzenie mobilne. W projekcie wykorzystano OpenXR, XR Hands, Meta XR SDK, passthrough, mikrofon, rozpoznawanie mowy Wit.ai oraz głos dyspozytora wygenerowany przy użyciu syntezy mowy Microsoft Paulina.

Kategoria	Technologia / zasób	Zastosowanie
Silnik	Unity 6 LTS	implementacja scen, logiki i build APK
Język	C#	skrypty, detektory, zarządzanie scenariuszem
XR	OpenXR, XR Hands, Meta XR SDK	śledzenie dłoni, passthrough, obsługa Quest
Sprzęt	Meta Quest 3 / 3S	uruchomienie aplikacji MR/VR
Głos	Microphone + Wit.ai	rozpoznawanie formułek w module 112
Audio	Microsoft Paulina TTS	głos dyspozytora
Modele	Mixamo, Sketchfab, Itch.io	intruz, auto, pokój, ciężarówka
Dźwięki	Freesound.org	telefon, syreny, uderzenie, dźwięki grozy
Analiza	JASP	planowana analiza danych z badania
Logowanie	JSONL	zapis zdarzeń, czasów i wyników

11. Funkcjonalność programu

Program realizuje funkcje podstawowe, kognitywne, badawcze i administracyjno-techniczne:

- Funkcje podstawowe – uruchomienie aplikacji, wybór trybu, przejście sesji, restart po przegranej i odtworzenie dialogów.
- Funkcje kognitywne – wsparcie decyzji i zachowania poprzez mikro kroki, feedback oraz trening pod presją.
- Funkcje badawcze – logowanie zdarzeń i przygotowanie do kwestionariuszy.
- Funkcje techniczne – budowanie scen z kodu, konfigurację manifestu Androida i generowanie APK.

Typ funkcji	Przykłady w CrisisReady
Podstawowe	menu, GRAJ, TRENING, restart, przejście pełnej sesji
Kognitywne	trening reakcji, kontrola ciszy, jasna komunikacja, krótkie feedbacki
Sensoryczne	hand tracking, detekcja pozycji głowy, mikrofon, passthrough
Decyzyjne	zaliczenie lub niezaliczenie mikro kroku, przejście do kolejnej fazy
Feedback	głos dyspozytora, dźwięki, jumpscare, ekran końcowy
Badawcze	metrics.jsonl, znaczniki czasu, wyniki, próby, błędy
Techniczne	builder scen, build APK, konfiguracja OpenXR i AndroidManifest

12. Logowanie danych i przygotowanie walidacji

Jednym z wymagań projektu było przygotowanie systemu do pomiaru efektywności. **CrisisReady** zapisuje metryki do pliku metrics.jsonl. Dane te mogą obejmować znaczniki czasu, nazwę aktu, nazwę mikrokroku, rezultat wykonania, liczbę prób, błędy i ukończenie scenariusza. Taki format jest prosty, czytelny i możliwy do późniejszej analizy w arkuszu kalkulacyjnym, Pythonie lub JASP.

Przykładowy fragment pliku metrics.jsonl (jeden wpis JSON w każdej linii):

```
{ "ts": "2026-06-09T18:42:03Z", "elapsed": 12.4, "session": "s01", "type": "microstep_started", "scenario": "act2_skid", "step_id": "grip_wheel", "index": 3 }
{ "ts": "2026-06-09T18:42:09Z", "elapsed": 18.1, "session": "s01", "type": "microstep_completed", "step_id": "grip_wheel", "result": "pass", "response_time_ms": 5700, "hints_count": 0 }
{ "ts": "2026-06-09T18:42:55Z", "elapsed": 64.3, "session": "s01", "type": "act2_crash" }
{ "ts": "2026-06-09T18:43:40Z", "elapsed": 109.0, "session": "s01", "type": "act3_thief_appears" }
{ "ts": "2026-06-09T18:44:25Z", "elapsed": 154.0, "session": "s01", "type": "act3_outcome", "result": "win", "silence_seconds": 45 }
```

Zmienna w logach	Znaczenie badawcze
participant_id	pseudonimowy identyfikator uczestnika
session_id	numer lub kod sesji badawczej
timestamp	czas zdarzenia
scenario_phase	akt lub moduł scenariusza
microstep_id	konkretny mikrokrok
result	sukces, błąd, przekroczenie czasu, przerwanie
response_time_ms	czas reakcji
hints_count	liczba podpowiedzi
noise_threshold_exceeded	czy przekroczono próg ciszy
speech_keywords_detected	czy wykryto wymagane słowa kluczowe

13. Proces wytwórczy, harmonogram i metodyka pracy

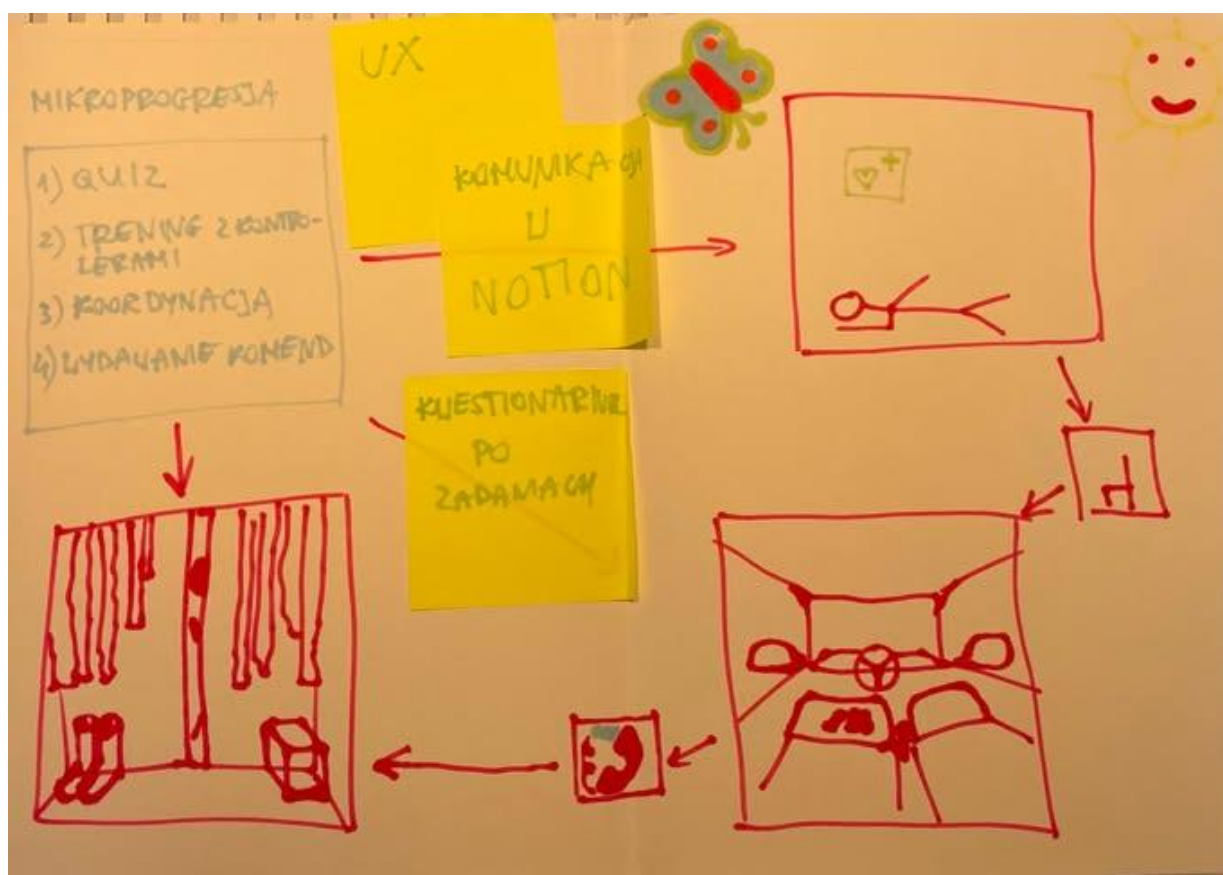
Prace prowadzono iteracyjnie. Zaczęliśmy pracę od analizy wymagań i artykułu wzorcowego, następnie przygotowaliśmy wstępną koncepcję scenariusza (zdjęcie poniżej), projekt UI i mechanikę mikrokroków. Kolejne etapy obejmowały implementację scen, integrację hand trackingu, rozpoznawania mowy, assetów i logowania, a następnie testowanie na fizycznych goglach Quest 3S. Ze względu na specyfikę VR/MR każda istotna zmiana wymagała budowania aplikacji, wgrania jej na gogle i sprawdzenia w rzeczywistych warunkach.

Etap	Zakres prac	Rezultat
1. Koncepcja	analiza problemu, wybór scenariuszy, decyzja o mikroprogresji	opis celu i głównych modułów
2. UX/UI	menu, trening, komunikaty, nazwy modułów	czytelne ekrany i uproszczony interfejs
3. Implementacja bazowa	sceny, akty, sterowanie, podstawowe przejścia	minimalny prototyp
4. Integracja MR/VR	passthrough, VR, hand tracking, mikrofon	działanie na Quest 3S
5. Integracja narracji	wypadek, intruz, telefon, syreny, dialog	spójny scenariusz fabularny
6. Testy i poprawki	debugowanie modeli, UI, wydajności i dłoni	grywalna wersja od początku do końca
7. Dokumentacja	opis systemu, role, problemy, bibliografia	sprawozdanie i materiały do oceny

Metodyka pracy była zbliżona do podejścia przyrostowego. Zamiast czekać na pełną implementację, testowaliśmy małe przyrosty: pojedynczy ekran, pojedynczą interakcję, pojedynczy akt lub pojedynczy detektor. Dzięki temu błędy techniczne były wykrywane względnie wcześniej, choć część problemów, takich jak śledzenie dłoni lub pozycjonowanie assetów, wymagała wielu iteracji.

Wspólny postęp był śledzony dzięki harmonogramowi w aplikacji Notion, gdzie każdy mógł kontrolować listę swoich zadań oraz termin na ich wykonanie, a także mógł zaznaczać, które z nich udało się już wykonać. Efekty swoich prac zamieszczaliśmy we wspólnym dysku Google – każdy miał swój folder, w którym umieszczał wykonane pliki, znalezione informacje czy linki oraz wszystkie potrzebne materiały. W ten sposób jako zespół byliśmy świadomi postępu, jaki jest dokonywany oraz listy zadań, które wymagają jeszcze pracy.

Wstępna koncepcja scenariusza:



14. Podział obowiązków w zespole

Projekt został zrealizowany przez pięcioosobowy zespół. Podział obowiązków obejmował: implementację mechaniki, testowanie, scenariusze dialogowe, dostarczanie assetów, oprawę graficzną, przygotowanie kwestionariuszy i analizę artykułu źródłowego.

- Jakub Cieniuch – mechanika gry, praca z Claude Code, assety, debugowanie, testy, implementacja logiki i integracja wielu komponentów
- Wiktoria Bartek – testowanie gry i feedback, wykrywanie błędów, sprawdzanie czytelności doświadczenia, przygotowanie końcowej prezentacji projektu
- Oliwia Frueauff – scenariusze i formułki dyspozytora 112, opracowanie treści komunikacji kryzysowej, opracowanie testów wiedzy związanych z poprawnym zachowaniem podczas poślizgu oraz resuscytacji na podstawie oficjalnych wymogów
- Magdalena Grzesiak – modele 3D i dźwięki, dostarczanie zasobów wizualnych i audio
- Adriana Jankowiak – oprawa graficzna menu i treningu, kwestionariusze, analiza MR.Drum, warstwa UI, badawcza i teoretyczna, opracowanie sprawozdania

Dodatkowo, każdy uczestnik miał aktywny twórczy wkład w projekt - od początku do końca brał udział w dyskusjach i burzach mózgu, dzielił się pomysłami i wpływał na końcowy efekt programu.

15. Testowanie techniczne

Obszar testów	Co sprawdzić	Kryterium akceptacji
Wydajność	FPS, spadki klatek, płynność renderowania	stabilne działanie bez zauważalnych przycięć
Opóźnienia	latency gest - reakcja, latency audio	feedback pojawia się wystarczająco szybko
Hand tracking	utrata śledzenia, jitter, warunki oświetlenia	dłonie są wykrywane w typowych warunkach pokoju
Mikrofon	uprawnienia, poziom szumu, wykrywanie ciszy	system poprawnie odróżnia ciszę od hałasu
Rozpoznawanie mowy	Wit.ai, słowa kluczowe, błędy transkrypcji	formułki 112 są rozpoznawane w większości prób
Pozycjonowanie assetów	skala, obrót, zapadanie w podłoże	modele są widoczne i nie psują immersji
Bezpieczeństwo	możliwość przerywania, seated mode	użytkownik nie musi fizycznie chodzić ani ryzykować kolizji
Logowanie	kompletność metrics.jsonl	po sesji można odtworzyć przebieg zadania

W prototypie wykonano testy na fizycznych goglach Quest 3S przez sideloading pliku APK. Każda większa zmiana wymagała ponownego builda i instalacji, co wydłużało iteracje. Testowanie obejmowało także casting obrazu z gogli, obserwację działania sceny i weryfikację podstawowych mechanik.

16. Propozycja badania użytkowników i procedura walidacji

Proponowana próba pilotażowa mogłaby obejmować 8-10 osób, co wystarcza do wykrycia głównych problemów UX i technicznych. W pełniejszym badaniu warto porównać grupę korzystającą z mikroprogresji MR/VR z grupą kontrolną, która otrzymuje klasyczny materiał szkoleniowy lub wersję VR bez adaptacyjnego feedbacku. Taki układ pozwalałby sprawdzić, czy przewagę daje sama immersja, czy właśnie pętla poznawcza i mikroprogresja.

Etap badania	Opis	Dane zbierane
Pre-test	uczestnik wykonuje uproszczone zadanie bez pełnego wsparcia	czas, błędy, liczba pominięć
Trening	uczestnik przechodzi tryb TRENING i sesję <i>CrisisReady</i>	czas mikrokroków, odpowiedzi, błędy
Post-test	uczestnik powtarza zadanie z mniejszą liczbą odpowiedzi	różnica względem pre-testu
Kwestionariusze	NASA-TLX, SUS, UEQ, IPQ, SSQ	obciążenie, użyteczność, obecność, cybersickness
Wywiad	krótkie pytania o trudność i zrozumiałość	jakościowe wyjaśnienie problemów
Retencja	powtórka po kilku dniach	utrzymanie efektu
Transfer	zmieniony wariant scenariusza	czy umiejętność nie jest tylko zapamiętaniem scenki

17. Metryki efektywności, UX i obciążenia poznawczego

W projekcie przewidziano trzy uzupełniające się grupy metryk, powiązane z pytaniami badawczymi (PB1–PB7) oraz ze zmiennymi rejestrowanymi w logach (zob. Tabela 9).

Metryki efektywności (behawioralne, z logów): czas do pierwszej poprawnej reakcji oraz czas ukończenia mikrokroku (`response_time_ms`), liczba błędów i restartów, liczba użytych podpowiedzi (`hints_count`) oraz ukończenie mikrokroków (`result`). Na ich podstawie wyznacza się prosty indeks mikroprogresji jako sumę punktów z mikrokroków w skali 0/1/2 (0 – brak wykonania, 1 – wykonanie z pomocą lub po czasie, 2 – wykonanie samodzielne) — wzrost indeksu po treningu oznacza przejście od działania niepewnego do samodzielnego.

Metryki UX: SUS (System Usability Scale) jako miara użyteczności oraz UEQ (User Experience Questionnaire) jako miara jakości doświadczenia. Metryki obciążenia poznawczego: NASA-TLX (subiektywne obciążenie zadaniem). Dodatkowo IPQ mierzy poczucie obecności, a SSQ – objawy choroby symulatorowej (cybersickness).

Na obecnym etapie (grywalny prototyp) zebrano dane techniczne i przygotowano kompletny zestaw zmiennych w logach; pełne wartości z udziałem użytkowników będą przedmiotem proponowanej procedury walidacji (zob. Tabela 12).

18. Wyzwania etyczne, prywatność i bezpieczeństwo

Projekt dotyczy sytuacji kryzysowych, stresu, głosu, ruchu i zachowania użytkownika, dlatego wymaga osobnej refleksji etycznej. Najważniejsze ryzyka dotyczą prywatności danych, bezpieczeństwa fizycznego, emocjonalnego obciążenia użytkownika, wtórnego użycia logów i nadmiernego zaufania do systemu.

Szczegółowe zestawienie ryzyk i środków zaradczych przedstawiono w Tabeli 13. Kluczowe znaczenie ma świadoma zgoda uczestników oraz możliwość przerywania sesji w dowolnym momencie. W warstwie danych stosuje się minimalizację: logi są pseudonimizowane (participant_id, bez imion i nazwisk), a nagrania głosu nie są przechowywane — zapisywany jest jedynie wynik rozpoznania lub wykryte słowa kluczowe.

Przetwarzanie danych powinno być zgodne z RODO, z preferencją dla lokalnego przetwarzania na urządzeniu i jasną informacją o celu oraz okresie przechowywania danych. Ze względu na obecność wypadku, włamania i jumpscare przewidziano ostrzeżenie o treści oraz krótki debriefing po sesji.

Ryzyko	Opis	Minimalizacja
Dane głosowe	moduł MOWA korzysta z mikrofonu i rozpoznawania mowy	nie zapisywać nagrań, tylko wynik rozpoznania lub słowa kluczowe
Dane behawioralne	logi opisują reakcje, błędy i czas użytkownika	pseudonimizacja participant_id, brak imion i nazwisk w logach
Stres i dyskomfort	scenariusz zawiera wypadek, włamanie i jumpscare	zgoda świadoma, możliwość przerywania, ostrzeżenie przed treścią
Cybersickness	VR/MR może powodować nudności lub dezorientację	SSQ, krótka sesja, seated mode, przerywanie na życzenie
Bezpieczeństwo fizyczne	użytkownik ma gogle na głowie i ograniczoną percepcję otoczenia	tryb stojąco/siedzący, brak chodzenia, clear play area
Overtrust	użytkownik może traktować system jako pełne szkolenie ratunkowe	jasny komunikat, że to prototyp treningowy, nie certyfikacja
Reidentyfikacja	połączenie logów z innymi danymi może ujawnić osobę	oddzielenie danych badawczych od kontaktowych, szyfrowanie
Wpływ emocjonalny	scenariusz może uruchomić lęk lub wspomnienia	kryteria wykluczenia, debriefing, możliwość odmowy udziału

19. Wykorzystanie AI

W projekcie wykorzystano narzędzia oparte na sztucznej inteligencji, jednak ich rola miała charakter wspierający, a nie zastępujący pracę zespołu. Szczególnie istotne było użycie Claude Code przy implementacji oraz poprawianiu fragmentów mechaniki gry. Narzędzie to wspierało pisanie kodu, rozwiązywanie problemów technicznych i usprawnianie wybranych funkcji, jednak każda zmiana była następnie testowana przez zespół na fizycznych goglach Meta Quest 3S. Ostateczne decyzje dotyczące działania mechanik, ich poprawności oraz użyteczności były podejmowane przez członków zespołu na podstawie testów, debugowania i obserwacji działania aplikacji.

ChatGPT został wykorzystany przede wszystkim do uporządkowania struktury raportu oraz redakcji tekstu sprawozdania. AI pełniło funkcję pomocniczą w organizacji i opracowaniu dokumentacji, ale nie było źródłem niezwyfikowanych informacji o projekcie.

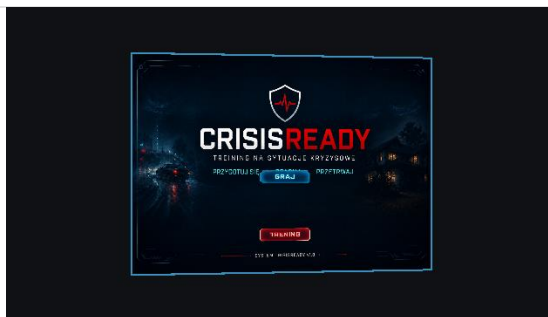
W module rozmowy z numerem 112 zastosowano Wit.ai (narzędzie do rozpoznawania wypowiedzi użytkownika). System umożliwiał identyfikowanie wybranych formułek i słów kluczowych wypowiedzianych przez gracza w sytuacji kryzysowej. Nasza kontrola polegała na samodzielnym dobraniu treści komunikatów, sprawdzeniu ich zrozumiałości oraz testowaniu, czy system poprawnie rozpoznaje najważniejsze elementy zgłoszenia. W razie problemów z rozpoznawaniem mowy nasz zespół modyfikował formułki i słowa kluczowe tak, aby lepiej pasowały do scenariusza.

Dodatkowo wykorzystaliśmy także syntezę mowy TTS Microsoft Paulina do wygenerowania głosu dyspozytora numeru 112. Pozwoliło to stworzyć bardziej realistyczny dialog głosowy w trakcie ćwiczenia MOWA. Wygenerowane kwestie zostały sprawdzone pod kątem zrozumiałości, naturalności oraz dopasowania do przebiegu scenariusza.

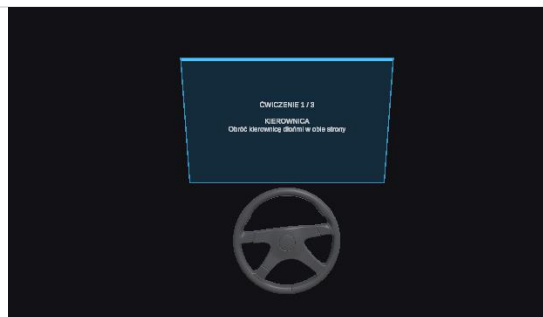
Narzędzia AI i automatyzacji zostały użyte jako wsparcie procesu projektowego, technicznego i redakcyjnego, natomiast odpowiedzialność za decyzje projektowe, testowanie oraz końcową jakość rozwiązania pozostała po stronie naszego zespołu.

20. Screeny, diagramy i dokumentacja działania

Poniżej zestawiono zrzuty ekranu z działającego prototypu na goglach Meta Quest, dokumentujące kolejne etapy sesji. Każdy rysunek opatrzono numerem i podpisem (co przedstawia i w jakim momencie scenariusza).



Rys. 1 — menu główne; wybór GRAJ/TRENING dłonią (bez kontrolerów).



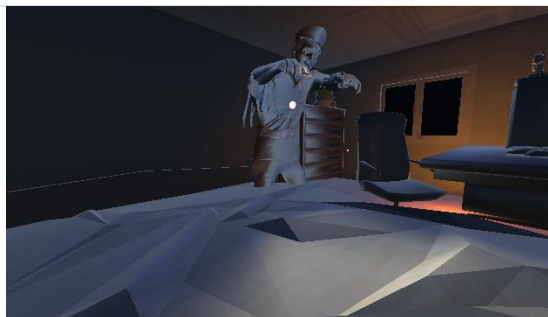
Rys. 2 — tryb TRENING, ćwiczenie KIEROWNICA przed właściwą grą.



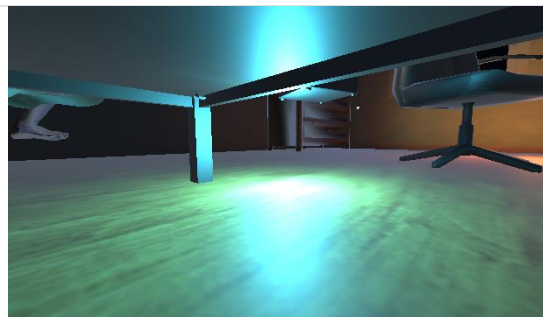
Rys. 3 — Akt II, jazda przez las z perspektywy kokpitu (passthrough/VR).



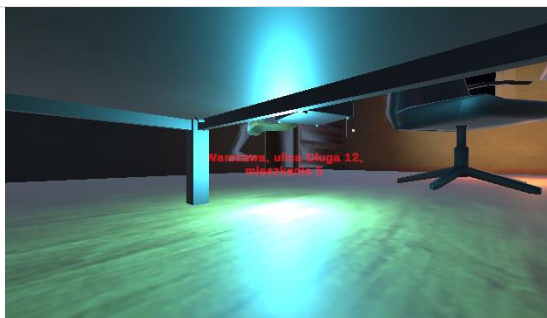
Rys. 4 — Akt II, wypadek — ciężarówka nadjeżdża z naprzeciwka.



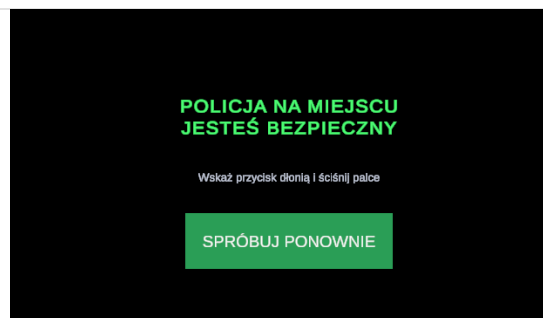
Rys. 5 — Akt III, intruz wchodzi do sypialni — moment zagrożenia.



Rys. 6 — Akt III, ukrycie pod łóżkiem; świecący telefon do zgłoszenia.



Rys. 7 — Akt III, zgłoszenie na 112 — kwestia gracza (moduł MOWA).



Rys. 8 — ekran końcowy — wygrana po przyjeździe policji.

Diagram pętli poznawczej (SENSE – INTERPRET – DECIDE – ACT – FEEDBACK) oraz architektury warstwowej omówiono w odpowiednich sekcjach i tabelach (Tabela 5 i opis architektury).

21. Niepowodzenia, ograniczenia i wnioski z błędów

Podczas realizacji projektu nasz zespół napotkał szereg problemów technicznych i projektowych, które wpłynęły na sposób dalszego rozwijania aplikacji. Jednym z pierwszych wyzwań była sama instalacja gry na goglach Meta Quest. Proces ten okazał się bardziej złożony niż standardowe uruchomienie aplikacji na komputerze, ponieważ wymagał założenia konta deweloperskiego Meta, włączenia trybu developer, autoryzacji gogli przez USB oraz ręcznego sideloadingu pliku APK. W praktyce oznaczało to, że każda większa zmiana w projekcie musiała zostać zbudowana, wgrana na urządzenie i ponownie przetestowana. W kolejnych etapach pracy należałoby więc już od początku planować dodatkowy czas na buildy, instalację oraz testy sprzętowe.

Pierwotny scenariusz gry zakładał trzy sytuacje kryzysowe: udzielenie pierwszej pomocy, wyprowadzenie pojazdu z poślizgu, wezwanie pomocy i zachowanie ciszy w ukryciu przed napastnikiem. Implementacja pierwszej minigry okazała się utrudniona z wielu względów. Pierwsze ograniczenie wiązało się z przeprowadzeniem resuscytacji przy użyciu kontrolerów. Poprawne przeprowadzenie resuscytacji wiąże się m.in. z poprawnym ułożeniem dłoni, co zostaje uniemożliwione przez kontrolery, dlatego postanowiliśmy zrezygnować z ich użycia. Jednak samo śledzenie dłoni przez program, bez kontrolerów również okazało się niemałym wyzwaniem. Nawet gdy śledzenie było możliwe, było ono na tyle nieprecyzyjne, że sytuacja udzielania pierwszej pomocy była bardzo nierealistyczna. To, co również wpływało na brak poczucia realizmu w tej scenie to fakt, że użytkownik przeprowadzał resuscytację w powietrzu, więc nie napotykał oporu. To dodatkowo zniekształcało odczuwanie rzeczywistości i zaburzało efekt naszej aplikacji. Jako że celem naszej gry było, aby użytkownik nie potrzebował niczego oprócz gogli VR, zrezygnowaliśmy z wykorzystania dodatkowych rekwizytów w świecie rzeczywistym, które mogłyby imitować reanimowanego człowieka (np. fantom czy poduszka). Końcowo, problemy ze skutecznym zaplanowaniem, implementacją oraz osiągnięciem oczekiwanego efektu spowodowały, że postanowiliśmy zrezygnować całkowicie z tej sceny – dlatego końcowa forma gry nie wykorzystuje sytuacji przeprowadzania RKO.

Istotnym problemem było sterowanie wyłącznie za pomocą dłoni. Z jednej strony hand tracking zwiększał naturalność interakcji i pozwalał zrezygnować z kontrolerów, ale z drugiej strony okazał się mniej stabilny niż klasyczne sterowanie. Śledzenie dłoni bywało zawodne szczególnie przy słabszym oświetleniu, nietypowej pozycji rąk lub zasłonięciu dłoni przez inne

obiekty. Zespół musiał więc wielokrotnie testować aplikację w różnych warunkach i dostosowywać mechaniki tak, aby były bardziej odporne na chwilową utratę trackingu.

Dużo pracy wymagało również poprawne ustawienie modeli 3D w scenie. Część assetów po zaimportowaniu miała niewłaściwą skalę, orientację lub punkt zaczepienia, przez co obiekty zapadały się w podłogę, lewitowały albo były obrócone w niewłaściwą stronę. Dotyczyło to między innymi elementów pokoju, łóżka, ciężarówki, telefonu czy postaci intruza. Każdy model musiał być ręcznie sprawdzany i dostosowywany do sceny, co wydłużało proces tworzenia. Z perspektywy dalszego rozwoju projektu ważne jest, aby przed włączeniem nowego assetu do gry sprawdzać jego pivot, skalę, kierunek ustawienia oraz zgodność z jednostkami używanymi w Unity.

Następnym wyzwaniem była wydajność aplikacji na goglach Quest 3S, ponieważ urządzenie działa samodzielnie i ma ograniczoną moc obliczeniową w porównaniu z komputerem, utrzymanie płynności rozgrywki wymagało optymalizacji sceny. Szczególnie obciążające były modele 3D, tekstury, cienie, efekty dźwiękowe oraz funkcja passthrough, która łączy obraz rzeczywistego pomieszczenia z elementami wirtualnymi. Aby uniknąć spadków liczby klatek na sekundę, konieczne było ograniczanie złożoności assetów, redukcja efektów wizualnych oraz testowanie działania aplikacji bezpośrednio na urządzeniu, a nie tylko w edytorze Unity.

Problemem okazała się także obsługa mikrofonu i rozpoznawania głosu. Moduł rozmowy z numerem 112 wymagał dostępu do mikrofonu, odpowiednich uprawnień runtime oraz dobrej jakości sygnału dźwiękowego. W praktyce rozpoznawanie wypowiedzi użytkownika mogło być zakłócanie przez hałas, niepewną wymowę, zbyt ciche mówienie albo problemy techniczne z dostępem do mikrofonu. Nasz zespół musiał więc dobrać takie formułki i słowa kluczowe, które były możliwe do rozpoznania przez system.

Osobnym wyzwaniem była naturalność dialogu z dyspozytorem numeru 112. Początkowo rozmowa brzmiała miejscami sztucznie lub zbyt mechanicznie, co mogło osłabiać realizm sytuacji. Nasz zespół musiał więc uprościć kwestie, dobrać je tak, aby pasowały do scenariusza oraz sprawdzić, czy są zrozumiałe dla użytkownika.

Istotnym ograniczeniem projektowym był również tryb siedzący Mixed Reality. Scenariusz zakładał między innymi ukrycie się pod łóżkiem, jednak użytkownik nie mógł realnie chodzić po pokoju ani fizycznie wejść pod wirtualne łóżko. Zespół musiał więc znaleźć rozwiązanie zastępcze i powiązać ukrycie z pozycją głowy użytkownika. Dzięki temu

zachowano sens scenariusza, ale dostosowano go do ograniczeń bezpieczeństwa, przestrzeni oraz trybu siedzącego.

Trudności pojawiły się także w obszarze interfejsu użytkownika. Klikanie elementów menu samą dłonią było mniej dokładne niż używanie kontrolera, dlatego małe przyciski lub zbyt precyzyjne interakcje okazywały się problematyczne. W odpowiedzi na to należało zwiększać elementy interfejsu, stosować promień wychodzący z palca oraz dodawać większą tolerancję błędów przy wyborze opcji.

Ostatnim problemem technicznym był ekran końcowy, który w niektórych sytuacjach prześwitywał przez geometrię sceny lub nie wyświetlał się w pełni poprawnie. Było to związane z kolejnością renderowania oraz sposobem osadzenia elementów UI w przestrzeni świata. Problem ten pokazał, że w aplikacjach VR i MR ekranów końcowych oraz komunikatów nie należy traktować tak samo jak klasycznego interfejsu 2D.

Podsumowując, większość napotkanych problemów wynikała z pracy na rzeczywistym sprzęcie VR/MR, a nie tylko z samego projektowania aplikacji w Unity. Najważniejszą lekcją z procesu tworzenia *CrisisReady* jest to, że projekty Mixed Reality wymagają ciągłego testowania na urządzeniu docelowym, uwzględnienia ograniczeń trackingu, wydajności i przestrzeni użytkownika oraz projektowania interakcji od początku z myślą o konkretnym sposobie sterowania.

22. Upowszechnienie projektu

Projekt został udostępniony za pośrednictwem repozytorium GitHub, które zawiera kod źródłowy aplikacji oraz dokumentację wraz z instrukcją niezbędną do włączenia programu. W przyszłości wraz z rozwojem projektu przygotowalibyśmy film demonstracyjny prezentujący działanie aplikacji VR, opublikowany w serwisie YouTube. Gotowa wersja programu zostałaby udostępniona do pobrania poprzez Google Drive, dzięki czemu byłaby łatwo dostępna dla wszystkich użytkowników, niezależnie od znajomości platformy GitHub.

Dokumentacja obejmuje opis projektu, instrukcję instalacji, uruchomienia oraz wymagania sprzętowe dotyczące gogli VR.

23. Dalszy rozwój systemu

CrisisReady może być rozwijane w kilku wzajemnie uzupełniających się kierunkach: technicznym, badawczym, użytkowym, scenariuszowym oraz edukacyjnym.

Istotnym kierunkiem rozwoju byłoby rozbudowanie elementów opartych na sztucznej inteligencji. System mógłby w przyszłości nie tylko rozpoznawać konkretne słowa kluczowe, ale także lepiej interpretować intencję wypowiedzi użytkownika, na przykład oceniać, czy zgłoszenie alarmowe zawiera najważniejsze informacje: lokalizację, opis zdarzenia, liczbę poszkodowanych i rodzaj zagrożenia. Możliwe byłoby także wprowadzenie adaptacyjnych podpowiedzi oraz personalizacji poziomu trudności. Aplikacja mogłaby dostosowywać presję czasu, liczbę wskazówek lub intensywność bodźców stresowych do doświadczenia i postępów użytkownika.

Z perspektywy badawczej kolejnym etapem powinna być pełna walidacja projektu na większej grupie uczestników. Dodatkowo można byłoby porównać grupę korzystającą z *CrisisReady* z grupą kontrolną, która otrzymuje tradycyjną instrukcję tekstową lub ogląda film szkoleniowy. Szczególnie ważnym elementem byłby także test transferu, czyli sprawdzenie, czy umiejętności wyćwiczone w środowisku MR/VR przenoszą się na bardziej realistyczne sytuacje poza aplikacją.

Projekt można byłoby również rozwijać pod kątem UX. W dalszych wersjach warto poprawić czytelność interfejsu, powiększyć przyciski, uprościć sposób interakcji dla osób bez doświadczenia z VR oraz dodać krótką kalibrację użytkownika przed rozpoczęciem scenariusza. Taki tryb wprowadzający mógłby wyjaśniać, jak działa śledzenie dłoni, w jaki sposób wybiera się opcje w menu i jak należy reagować na komunikaty w grze. Dzięki temu aplikacja byłaby bardziej dostępna dla dzieci, osób starszych lub użytkowników, którzy po raz pierwszy korzystają z gogli VR.

Największy potencjał rozwojowy dotyczy jednak rozbudowy scenariuszy. *CrisisReady* można byłoby rozszerzyć o kolejne sytuacje kryzysowe, takie jak udzielanie pierwszej pomocy, pożar, ewakuacja z budynku, zagubienie się dziecka, załabnięcie osoby w miejscu publicznym, agresywne zachowanie obcej osoby czy różne warianty wypadków drogowych. Obecny scenariusz poślizgu samochodu mógłby zostać rozwinięty w moduł przeznaczony dla szkół jazdy, gdzie kursanci mogliby bezpiecznie trenować reakcje na utratę kontroli nad pojazdem, nagłe wtargnięcie przeszkody na drogę albo konieczność wykonania manewru awaryjnego. Z kolei moduły dotyczące rozmowy z numerem alarmowym i zachowania ciszy mogłyby zostać

zaadaptowane jako narzędzie edukacyjne dla szkół, uczące dzieci i młodzież, jak zachowywać się w sytuacjach zagrożenia, jak rozmawiać z dyspozytorem oraz jak opanować stres.

W dłuższej perspektywie *CrisisReady* mogłoby stać się szerszą platformą treningową do nauki reakcji kryzysowych. Takie narzędzie mogłoby być wykorzystywane nie tylko w edukacji szkolnej, ale także w szkołach jazdy, na kursach pierwszej pomocy, w szkoleniach BHP, warsztatach bezpieczeństwa czy programach profilaktycznych. Zaletą takiego rozwiązania jest możliwość wielokrotnego przećwiczenia trudnych sytuacji w bezpiecznym środowisku, bez realnego narażania użytkownika na niebezpieczeństwo. Mixed Reality pozwala jednocześnie zachować element immersji i stresu sytuacyjnego, który jest trudny do osiągnięcia w tradycyjnych formach szkolenia.

Równolegle należałoby rozwijać kwestie etyczne i bezpieczeństwa danych. W przyszłych wersjach aplikacji konieczne byłoby opracowanie pełnego protokołu świadomej zgody, szczególnie jeśli system byłby testowany na dzieciach lub osobach niepełnoletnich. Należałoby również jasno informować użytkowników, jakie dane są zbierane, w jakim celu są analizowane i jak długo będą przechowywane. Preferowanym rozwiązaniem byłoby lokalne przetwarzanie danych, ograniczenie zapisu nagrań głosu oraz umożliwienie użytkownikowi kontroli nad ustawieniami prywatności. Dzięki temu rozwój *CrisisReady* mógłby łączyć innowacyjność technologiczną z odpowiedzialnym podejściem do edukacji, bezpieczeństwa i ochrony danych.

24. Wnioski końcowe

CrisisReady pokazuje, że możliwe jest stworzenie prototypu technologii kognitywnej w MR/VR, która nie ogranicza się do prezentowania sceny, lecz aktywnie wspiera użytkownika w uczeniu się reakcji kryzysowych. Projekt integruje trzy typy działania: motoryczne, komunikacyjne i samoregulacyjne. Każdy z nich został powiązany z mikrokrokami, możliwością feedbacku i metrykami do przyszłej walidacji.

Największą wartością projektu jest połączenie narracyjnego scenariusza z logiką mikroprogresji. Użytkownik nie wykonuje izolowanych ćwiczeń, ale przechodzi przez spójną sytuację kryzysową, w której wcześniejsze zdarzenia nadają sens późniejszym decyzjom. Jednocześnie system zachowuje strukturę badawczą: poszczególne kroki można mierzyć, porównywać i analizować.

Najważniejsze ograniczenia dotyczą stabilności technicznej, naturalności dialogu, warunków działania hand trackingu oraz braku pełnego badania użytkowników. Nie obniża to jednak wartości prototypu, ponieważ projekt osiągnął etap grywalnej demonstracji i zawiera fundamenty do dalszej walidacji.

Na obecnym etapie, wobec braku badania z udziałem użytkowników, postawione hipotezy (główna oraz szczegółowe) pozostają niezverifyfikowane empirycznie. System rejestruje jednak dokładnie te dane (czas reakcji, liczba błędów, liczba podpowiedzi i ukończenie mikrokroków — zob. Tabela 9), które pozwolą je przetestować w proponowanej procedurze walidacji (Tabela 12), odpowiadając na pytania badawcze PB1–PB7. Zrealizowano natomiast wszystkie kryteria sukcesu o charakterze implementacyjnym (Tabela 2): działający prototyp na Quest 3/3S, rozpoznawalne mikrokroki, działanie w pętli zamkniętej oraz logowanie danych do późniejszej analizy.

Bibliografia

- [1] Wang, X. i in. (2025). MR.Drum: Designing Mixed Reality Interfaces to Support Structured Learning Micro-Progression in Drumming.
- [2] Wood, D., Bruner, J. S., Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100.
- [3] Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.

- [4] Hart, S. G., Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index). *Human Mental Workload*, 139–183.
- [5] Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. *Usability Evaluation in Industry*.
- [6] Schubert, T., Friedmann, F., Regenbrecht, H. (2001). The experience of presence: Factor analytic insights (IPQ). *Presence*, 10(3), 266–281.
- [7] Kennedy, R. S. i in. (1993). Simulator Sickness Questionnaire (SSQ). *The International Journal of Aviation Psychology*, 3(3), 203–220.